

机器人避障寻址问题

实验报告

数据科学与大数据技术专业

2451951 于沛可

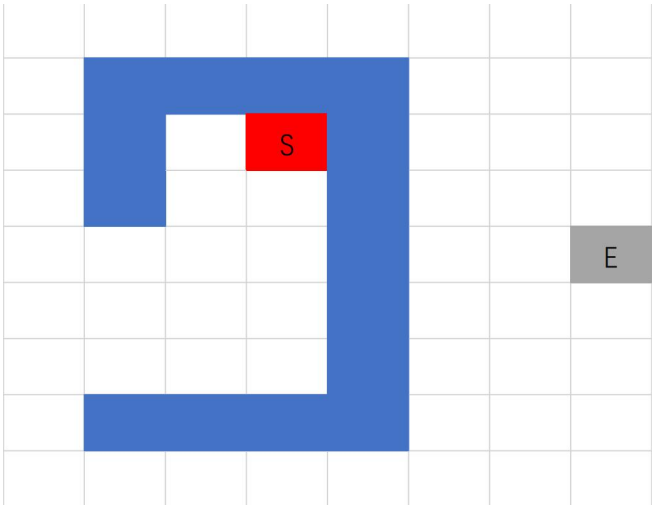
2026 年 4 月 15 日

一. 问题概述

1. 实验内容

机器人自主导航是智能机器人技术体系中的核心研究方向，而避障寻径作为自主导航的关键环节，直接决定了机器人在复杂环境中的作业能力。 本次实验以静态栅格地图下的机器人避障寻径为基础场景。在一个包含固定起点（S）、终点（E）及不可通行障碍区域的离散网格中，要求实现从起点到终点的最短路径搜索。实验不仅要求实现经典的 A* 与 BFS 算法，还需要通过动态可视化手段展示搜索过程，并进一步引入启发式算法（如人工势场法、强化学习法）和 Dijkstra 算法进行性能对比，深刻剖析各算法在空间搜索中的本质差异与局限性。

本实验中，给定如下的机器人避障寻址场景图：



其中 S 为机器人寻径的起点，E 为寻径的终点，蓝色区域为无法通过的障碍。要求机器人从 S 点出发，绕开障碍，到达 E 点。本实验需要参与者自行设计核心算法规则完成机器人从起点到终点的最短路径搜索。

2. 实验要求

本次实验的具体要求如下：

1. 给出场景矩阵表示，禁止硬编码（hard code），求解起点到终点的最短路径。

2. 使用 A* 和 BFS 算法，对起点到终点进行可视化搜索，并详细说明算法流程与原理。

3. 比较 A* 与 BFS 算法的差异和性能区别。

4. 自主设计更高难度场景图，结合启发式算法（如人工势场法或强化学习）进行求解与改进。

5. 提出其他不同的解决方案（如 Dijkstra 等），分析所设计算法的局限性及改进点。

6. 最短路径查找过程的动态化展示。

3. 背景知识：图搜索与启发式规划

在复杂的地图环境中寻找路径，本质上是一个图搜索问题。传统的搜索算法（如BFS、Dijkstra）属于“盲目搜索”或“无信息搜索”，它们按特定数据结构（如队列、最小堆）盲目地向外扩展状态空间，直到触碰目标边界。

为了应对大规模状态空间带来的维度灾难，人工智能领域引入了“启发式搜索”，其中最具代表性的便是 A* 算法。它在传统代价函数的基础上，引入了面向目标的预估代价，使得搜索过程具有了“方向感”。而人工势场法则进一步抛弃了全局状态树的维护，将搜索过程转化为在连续/离散势场中的梯度下降最优化问题。

4. 问题建模与思路分析

拿到给定的场景图后，我首先将物理环境抽象为数学模型：

状态空间映射：将地图定义为 $m \times n$ 的二维矩阵。矩阵元素 $grid[y][x]$ 值为 0 表示可通行自由空间，值为 1 表示不可通行障碍物。

动作空间定义：设定机器人为质点模型，单次只能进行上、下、左、右四个正交方向的状态转移，禁止对角线穿透。

代价函数：基于正交移动的设定，这是一张等权图，即每发生一次合法状态转移，单步路径代价 $C = 1$。

基于以上建模，寻找最短路径等价于在该加权网格图中寻找起始节点到目标节点代价和最小的节点序列。

二. 算法设计

本节将深入探究不同路径规划算法的底层控制逻辑与状态扩展机制。

1. 广度优先搜索 (BFS)：基于状态空间树的层序扩展

算法原理：BFS 利用 FIFO（先进先出）队列进行状态管理。它以起点为根节点，按搜索树的深度逐层生成和检查状态。

最优性保证：在单步代价恒为 1 的栅格地图中，节点在搜索树中的深度严格等价于其物理路径的代价。因此，当目标节点首次被扩展并进入队列时，系统处于当前的最小深度搜索层，这在数学上严格保证了找出的路径是全局最优解。

局限性：BFS 的状态扩展呈各向同性。随着深度的增加，其需要维护的中间状态（队列长度）呈指数级增长，空间复杂度为 $O(b^d)$（b 为分支因子）。在开阔地图中，这会导致大量无用节点的内存占用和算力浪费。

2. A* 算法：基于优先队列的启发式最优搜索

A* 算法旨在解决 BFS 盲目扩展的效率瓶颈，其核心在于引入了评估函数：

$$f(n) = g(n) + \hbar\,(n)$$

$g(n)$：从起点到当前节点 n 的实际已花费代价。

$\hbar\,(n)$：从当前节点 n 到终点的启发式预估代价。

优先扩展机制：算法内部维护一个最小堆作为 Open Set。每次以 $O(\log N)$ 的时间复杂度弹出 $f(n)$ 最小的节点进行扩展，使其能够沿着高概率接近目标的梯度方向深入。

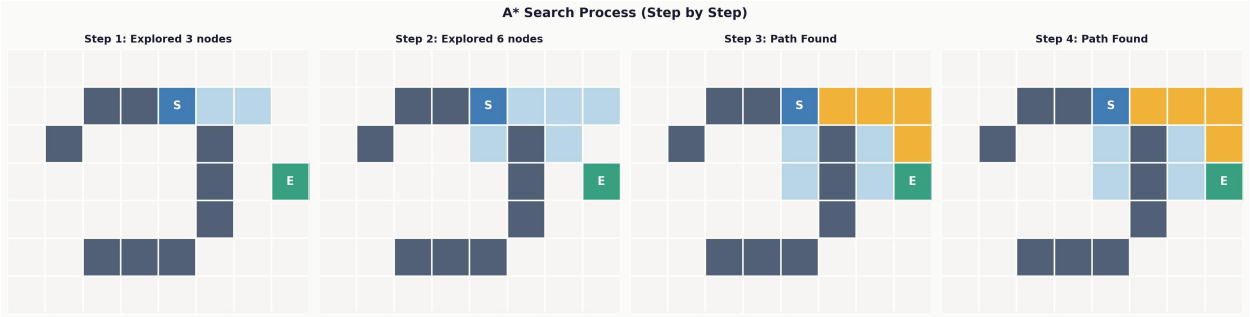
启发函数与可采纳性：A* 能否保证最优解，取决于 $\hbar\,(n)$ 是否满足可采纳性，即必须满足 $\hbar\,(n) \leq \hbar^*(n)$（预估代价永远不超过实际最小代价）。

由于本实验限制为四向正交移动，我选用了曼哈顿距离：

$$\hbar\,(n) = |x_n - x_E| + |y_n - y_E|$$

该距离完美贴合网格物理特性，严格不高于实际代价，在极大提升搜索剪枝效率的同时，保留了获取全局最优解的数学保证。

一个 A*算法动态探索具体例子可视化（非本题场景）：



3. Dijkstra 算法：A* 的退化形式

算法本质：Dijkstra 算法可以视作 A* 算法在启发信息 $h(n) \equiv 0$ 时的特例。它完全依赖累积代价 $g(n)$ 的大小进行贪心扩展。

性能分析：虽然它在非负权图中能保证最优，但在本实验的等权栅格中，由于缺乏方向引导，其优先队列的扩展等值线呈圆形向外辐射（类似 BFS）。它引入了堆排序的时间开销，却未能像 A* 一样缩减探索规模，在此场景下效率垫底。

4. 人工势场法：局部最优化的梯度下降

为了探索不同于传统图搜索（如 A*、BFS）的解法，本实验在自定义场景中引入了人工势场法（Artificial Potential Field, APF）。相比于 A* 需要维护庞大的搜索树，APF 具有极低的计算复杂度（单步决策 O(1)），但其局部最优化的特性也带来了明显的局限。

4.1 核心算法原理

APF 的本质是将路径规划问题转化为在虚拟物理场中的运动问题。算法不需要全局视野，而是给地图定义两种“力场”：

引力场：由终点产生，距离终点越远，引力势能越大。它负责引导机器人向目标大方向移动。

斥力场：由障碍物产生，只有当机器人靠近障碍物到一定阈值内才会触发，距离越近斥力呈指数级增大，负责实时避障。

在代码的每一步执行中，机器人只需计算其周围相邻栅格的总势能（引力 + 斥力），然后像水往低处流一样，贪心地走向总势能最小的那个格子（即沿着合力方向也就是负梯度方向移动）。

4.2 致命缺陷：局部极小值陷阱

APF 的优势在于计算极快，但在面对复杂拓扑地图（例如本实验中设计的“U 型”或“凹字形”障碍物）时，存在严重的数学缺陷。

假设机器人朝向终点进入了一个 U 型槽内部，我们可以清晰地看到受力对消的过程：

侧向斥力抵消：U 型槽左右两边的墙壁对机器人产生侧向的斥力，因为对称性，这两股力相互抵消。

纵向拉扯死锁：终点在前方，试图把机器人往前拉；但 U 型槽的底部是一堵墙，距离越近向后的斥力越大。

陷阱触发：必然存在一个特定的临界位置，使得向前的引力与向后的斥力大小完全相等、方向相反。此时，机器人受到的合力为零（即系统总势能梯度为零）。机器人误以为找到了一个“稳定点”，陷入了非目标的局部极小值洼地中，丧失了继续寻路的动力。

4.3 离散地图中的高频震荡现象

在连续的数学模型中，合力为零会导致机器人停滞。但在本实验的二维栅格数组中，机器人每次只能移动一个整数格，这导致了更糟糕的工程现象——状态震荡。当机器人走到格子 A，算出的合力微弱向前，于是移动到前方的格子 B；但一到格子 B，离墙壁变近导致斥力瞬间飙升，合力变成向后，下一步算法又会把它推回格子 A。反映在可视化界面上，就是机器人陷入了死循环，在两个相邻格子之间高频“反复横跳”，彻底规划失败。

4.4 改进方向

通过实测现象可以看出，APF 虽然避障反应极快，但缺乏 A* 那种绕开死胡同的宏观视野。要解决极小值陷阱，工程上通常有两种改进方向：

引入随机扰动：监控路径状态，一旦发现机器人连续几步在极小范围内徘徊，就强行加入一个随机方向的力向量，把它“踹”出势能洼地。

与 A* 结合的混合架构：利用 A* 在稀疏的宏观地图上规划出几个关键的“途径点”，然后利用 APF 在这些点之间进行局部的平滑移动和动态避障。

5. 强化学习：基于马尔可夫决策过程的无模型寻径

为了摆脱对环境先验知识（如启发函数或地图拓扑）的依赖，本实验在高难度场景中引入了强化学习的基础算法。相比于图搜索算法的全局遍历，Q-Learning 赋予了智能体“自主试错”的能力，使其能够在完全未知的环境中学习出最优策略，但这种无模型的学习方式在离散空间中同样存在工程局限。

5.1 核心算法原理

Q-Learning 的本质是将路径规划转化为马尔可夫决策过程（MDP） 。算法不需要预先建立环境地图，而是维护一张 Q 表（Q-Table），记录在各个“状态（当前坐标）”下采取不同“动作（上下左右）”的预期累计收益。

奖励机制：驱动智能体学习的核心。系统规定：到达终点给予强正激励，撞墙越界给予强负惩罚，而每发生一次普通移动也会扣除微小分数（时间惩罚）。

价值更新：在代码的每次执行中，智能体以 ϵ-贪婪策略选择动作，利用环境反馈的即时奖励，结合贝尔曼方程不断更新 Q 表：

$$Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma \max_{a'} Q(s',a') - Q(s,a)]$$

经过大量迭代，Q 表收敛。推理时，机器人只需在当前格子上贪心地选择 Q 值最大的动作，即可连成最优路径。

5.2 致命缺陷：稀疏奖励下的盲目探索陷阱

Q-Learning 的优势在于推理极快（O(1)），但在面对高难度且空旷的迷宫时，其训练阶段面临着严重的稀疏奖励问题。

由于智能体初始状态下对环境一无所知，完全依靠随机游走探索。在它极度偶然地第一次踏入终点格子（获得巨大正奖励）之前，它收到的反馈全都是负面的。在庞大的状态空间中，这种“漫无目的的试错”会导致训练初期的收敛速度极其缓慢，在设定的迭代次数内极易出现无法收敛的情况。

5.3 离散地图中的碰壁与打转现象

在训练早期（此时探索率较高且 Q 表全为 0），会观察到非常典型的工程现象——碰壁与打转。

智能体很快发现“只要移动就会被扣分”。如果惩罚权重设置得不够精细，智能体会在逻辑上陷入误区：它宁愿在一个角落里左右来回打转，或者原地不断“撞墙”（虽然撞墙惩罚高，但若衰减因子 γ 或学习率 α 设定不当，局部预期收

益可能高于长途跋涉的累积惩罚）。反映在训练曲线上，即前数百个 Episode 的累计奖励在负数深渊中震荡，直到偶然命中终点，奖励信号才开始沿着轨迹回传。

5.4 改进方向

表格型 Q-Learning 极其缺乏方向感，且存在状态数量随地图变大呈指数爆炸的“维度灾难”。工程上通常有以下两种改进方向：

深度强化学习（Deep Q-Network, DQN）：引入深度神经网络来代替庞大的 Q 表。利用神经网络的非线性拟合能力，将坐标输入网络直接输出动作的 Q 值，从而应对超大规模寻径问题。

基于势能的奖励塑形 ：将 APF 的思想融入强化学习。不再只等到达终点才给奖励，而是每当智能体向目标靠近一步（如曼哈顿距离缩短），就给予一个微小的正向奖励，极大地加速网络收敛。

三. 实验结果

1.运行环境与代码模块

Python 版本：Python 3.10

主要依赖：Numpy, Matplotlib

本实验代码分为多个模块化文件，组织结构如下：

grid_map.py：地图矩阵定义及起点、终点坐标配置。

algorithms.py：包含 BFS, A*, Dijkstra 的核心算法实现。

advanced_algorithms.py：APF, Q-Learning 的核心算法实现

visualize.py：基于 matplotlib.animation 的搜索过程动态渲染模块。

main.py：汇总运行并对比输出。

2.基础场景：经典图搜索算法对比

在给定的基础网格地图中，设定起点为 (2, 3)，终点为 (4, 7)。A*（采用曼哈顿距离）、BFS 与 Dijkstra 算法的实测数据如下表所示：

算法模型	找到的路径步数	探索节点数 (ClosedSet)	执行耗时 (秒)	路径是否最优

算法模型	找到的路径步数	探索节点数 (ClosedSet)	执行耗时 (秒)	路径是否最优
A* (Manhattan)	20	40	0.000285	是
BFS	20	57	0.000204	是
Dijkstra	20	57	0.000249	是

实际搜索路径形态对比：

A* 路径：

(2, 3) → (3, 3) → (4, 3) → (4, 2) → (4, 1) → (4, 0) → (3, 0) →

(2, 0) → (1, 0) → (0, 0) → (0, 1) → (0, 2) → (0, 3) → (0, 4) →

(0, 5) → (0, 6) → (0, 7) → (1, 7) → (2, 7) → (3, 7) → (4, 7)

BFS 路径：

(2, 3) → (3, 3) → (4, 3) → (5, 3) → (6, 3) → (6, 2) → (6, 1) →

(6, 0) → (7, 0) → (8, 0) → (8, 1) → (8, 2) → (8, 3) → (8, 4) →

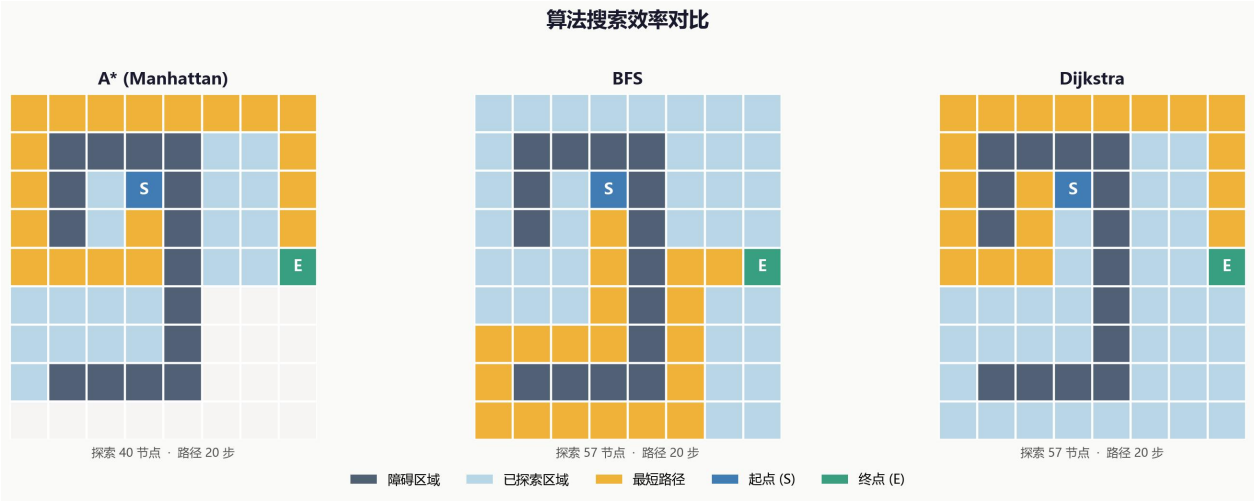
(8, 5) → (7, 5) → (6, 5) → (5, 5) → (4, 5) → (4, 6) → (4, 7)

Dijkstra 路径：

(2, 3) → (2, 2) → (3, 2) → (4, 2) → (4, 1) → (4, 0) → (3, 0) →

(2, 0) → (1, 0) → (0, 0) → (0, 1) → (0, 2) → (0, 3) → (0, 4) →

(0, 5) → (0, 6) → (0, 7) → (1, 7) → (2, 7) → (3, 7) → (4, 7)



算法探索详细动图如下：

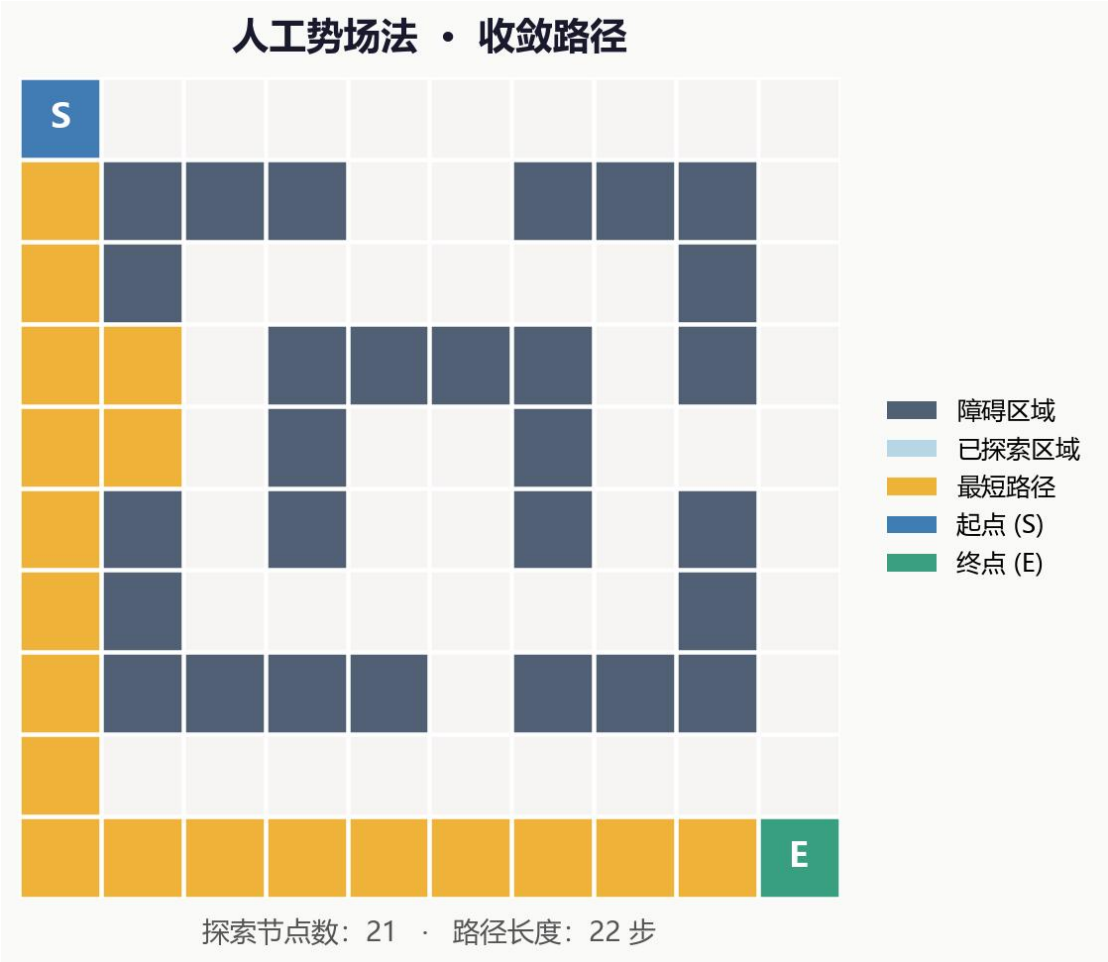
A*：astar_animation.gif

BFS：bfs_animation.gif

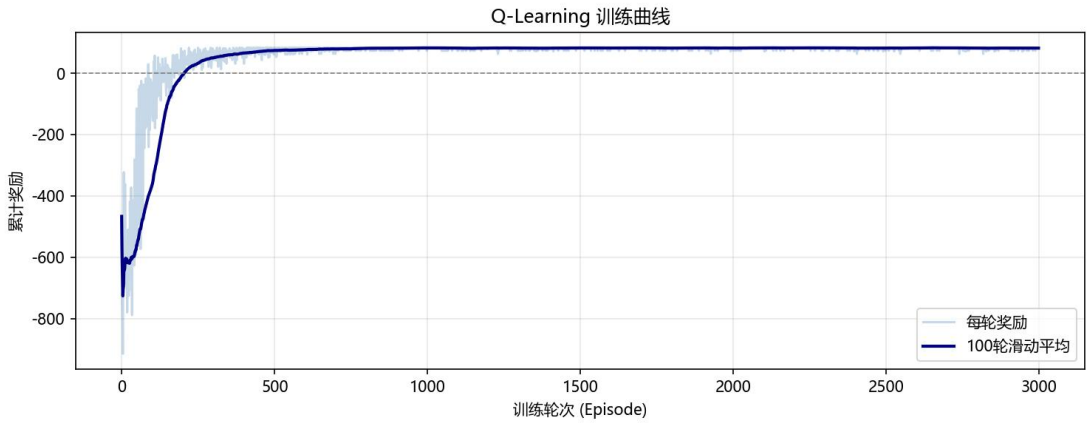
（动态搜索过程的 GIF 动画已作为附件与源代码一并打包提交，请查阅附件文件夹中的 astar_animation.gif 与 bfs_animation.gif）

2. 高难度场景：进阶启发式算法测试

为测试算法的泛化能力与鲁棒性，自行设计了带有“凹字形”陷阱的高难度场景。**人工势场法：**实测中，由于其局部贪心特性，算法虽然极快（执行耗时微秒级），但在绕过复杂障碍物时易贴墙滑行，最终收敛路径长度为 22 步，探索节点数为 21，未能达到严格意义上的全局最短路。



强化学习：共进行 3000 个 Episode 的训练，总耗时 0.58s。

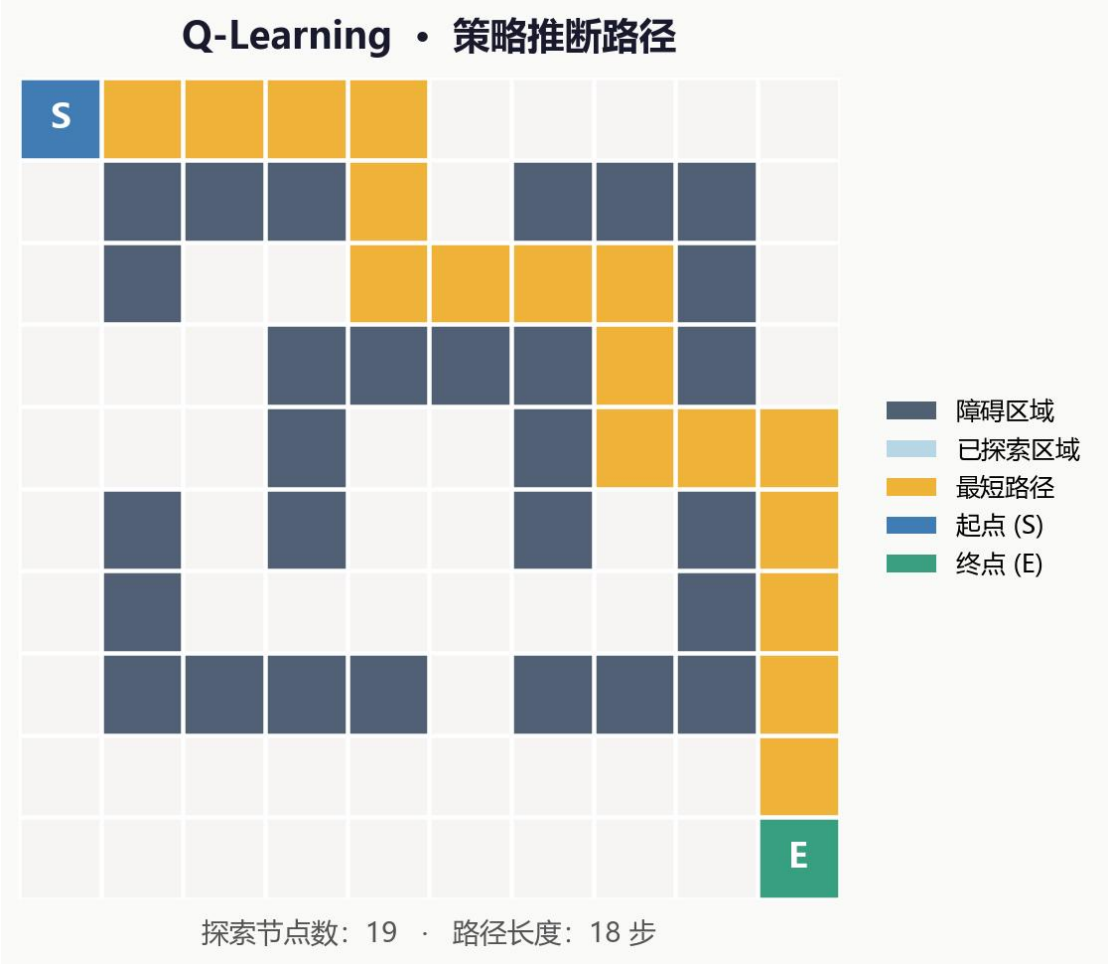


训练过程记录：

前 500 轮（ϵ =0.082）：智能体处于强探索阶段，频繁触发撞墙惩罚，近 500 轮平均奖励低至 -36.4。

第 1000 轮（ϵ =0.010）：Q 表初步收敛，智能体找到了通往终点的路径，平均奖励激增至 80.1。

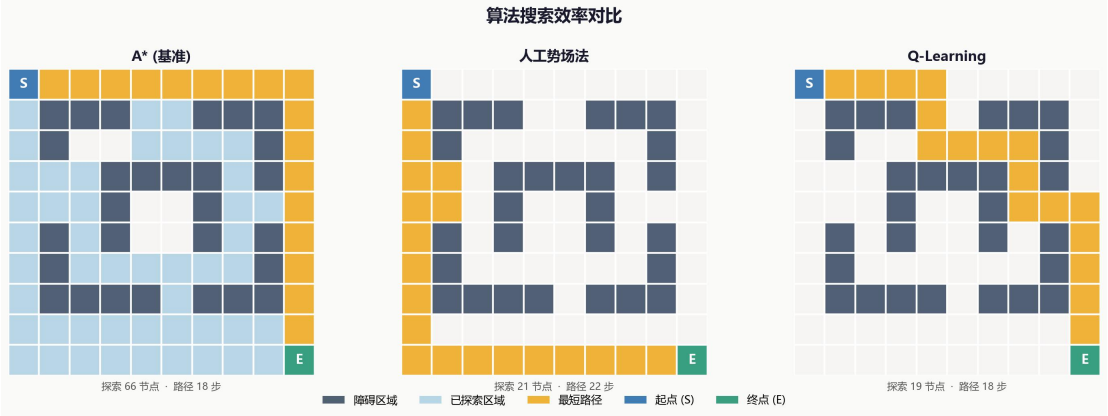
第 1500~3000 轮：平均奖励稳定在 82.1。最终策略推断成功生成了一条长度仅为 18 步的全局最优路径 。



对比：

算法模型	核心机制	寻路/推理耗时(s)	探索节点/迭代轮次	最终路径步数	是否全局最优
A*（基准参考）	全局启发搜索	< 0.001	66 个（Closed Set）	18	是
人工势场法(APF)	局部梯度下降	0.0335	21 个（无回溯轨迹）	22	否（次优解）

算法模型	核心机制	寻路/推理耗时 (s)	探索节点/迭代轮次	最终路径步数	是否全局最优
Q-Learning	MDP 价值迭代	0.5800（训练耗时）< 0.001（推理耗时）	3000 个 Episode	18	是



四. 结果分析

1. 启发式偏置的优势

在基础场景中，A*、 BFS 和 Dijkstra 均毫无悬念地找到了 20 步的全局最优解，但其内部的探索机制却截然不同。A* 算法的探索节点数（40 个）比 BFS 和 Dijkstra（均为 57 个）减少了近 30%。这直观地证明了曼哈顿距离作为目标导向偏置的强大威力。BFS 是无向的“水波纹”辐射，而 A* 则像一根带有磁性的针，强行将搜索树的展开方向压缩在起点与终点的对角线区域内。

2. APF 局部贪心的局限

在高难度的“凹字形”陷阱场景中，人工势场法（APF）的表现极具警示意义。它仅仅评估了 21 个物理节点，就极速收敛出了一条 22 步的路径，但未能达到 18 步的全局最优。

缺乏宏观拓扑视野的代价：APF 的本质是物理空间中的连续梯度下降。它不需要像 A* 那样维护庞大的 Open/Closed 集合，单步决策仅为 O(1)。但实测中出现的“贴墙滑行”现象表明，当连续的排斥力场被降维映射到只能上下左右移动的离散栅格时，极易产生力的对冲与抵消。

我的反思：APF 是一种纯粹的“短视”算法。它陷入局部极小值的根本原因，在于它没有状态回溯机制（无记忆性）。在真实工业场景（如自动驾驶）中，APF 绝不能单独作为全局规划器使用，它只适合作为 A* 等全局算法底层的一层动态避障平滑器，这就是“全局图搜索 + 局部势场避障”混合架构存在的必然性。

3.强化学习分析

强化学习在复杂场景中的表现，向我展示了路径规划从“搜索”向“学习”的范式转移。

观察训练过程，前 500 轮智能体的平均奖励深陷负值深渊（-36.4）。这是由于在无模型设定下，智能体根本不知道“墙壁”和“终点”的概念，完全依赖布朗运动式的随机试错。正是得益于 ϵ-贪婪策略中对 ϵ 值的科学衰减（从 0.082 降至 0.010），智能体在第 1000 轮度过了最艰难的稀疏奖励期（奖励跃升至 80.1），并在后续轮次收敛。

从量化对比表格可以看出，Q-Learning 花费了长达 0.58 秒进行 3000 次试错训练。如果单看这个指标，它被 A*（< 0.001s）完全碾压。但这引发了我对算法生命周期的深层思考——Q-Learning 的这 0.58 秒是“前置沉没成本”。一旦 Q 表（马尔可夫策略分布）收敛，其在实际运行中的“推理耗时”瞬间降至微秒级（查表复杂度 O(1)），且直接吐出 18 步全局最优路径。这证明了 RL 在处理高频重复任务时，具有传统搜索算法无法比拟的实时响应优势。

那么，什么时候更高级的算法可以体现出优势呢？

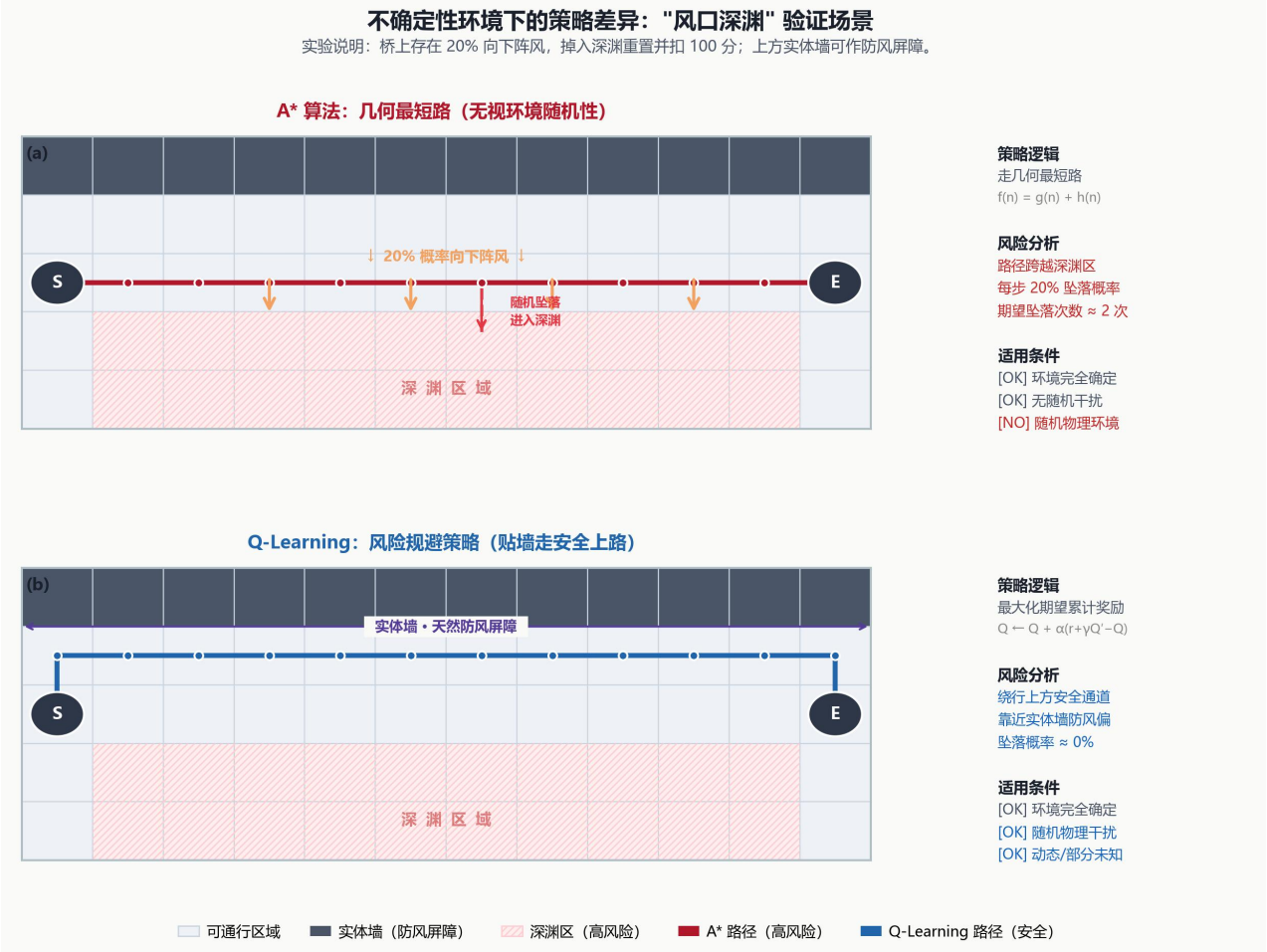
五．何时强化学习算法能体现优势？

既然 A* 如此高效，我们为何还需要 APF 或强化学习？答案在于算法对环境的假设前提。

传统图搜索（如 A*）是典型的“理想主义者”。它能在本实验中大杀四方，是因为环境满足三个极其苛刻的假设：绝对静态、全局已知、100% 确定性执行。但在真实的机器人工程（如四足机器人越野、无人机抗风飞行）中，我们下达“向前走一步”的指令，底盘极有可能因为打滑或风偏导致向侧后方偏移。 一旦引入物理不确定性，A* 这种只追求几何极致的算法就会瞬间崩溃，而强化学习追求数学期望收益最大化的底层优势才会真正爆发 。

拓展验证：“风口深渊”博弈 (Windy Abyss Bridge)

为了直观证实上述结论，本报告引入了一个极端测试场景： 地图中央有一条直达终点的捷径（深渊独木桥），但桥上常年刮着向下的侧风（存在 20% 概率将机器人吹向下方一格）。桥的下方是深渊（掉入扣 100 分），而地图边缘有一条漫长但受实体墙壁保护的安全通道。



从几何角度看，A* 会坚定选择中路捷径；但从期望收益看，这条路叠加了显著的随机灾难风险。相反，强化学习通过与环境交互，学习到“贴墙绕行”的安全策略，虽然路径更长，却显著降低了高代价失败事件，最终获得更高的长期期望回报。这说明在不确定环境中，“最短路径”不等于“最优策略”。

那么既然 A* 高效，为什么还需要 APF 或强化学习？因为它们解决的是 A* 难以覆盖的部分。APF 的优势在于局部实时反应，适合处理突发障碍、连续空间微调与即时避碰；强化学习的优势在于统计意义下的策略优化，尤其适用于存在噪声、模型不完备、奖励具有长期性的任务。工程上更常见的做法不是单一算法取代，而是分层融合：A* 负责全局可达性，APF 负责局部安全与平滑，强化学习负责不确定扰动下的策略自适应与鲁棒性提升。

因此，本实验的核心结论不是“A* 无效”，而是“算法有效性高度依赖环境假设”。在确定性静态环境里，A* 依然是高效且可靠的基线；在随机扰动、动态变化或部分未知环境中，强化学习与局部反应方法的价值才会真正体现

六．算法改进讨论与消融实验验证

为了严谨评估进阶算法在复杂环境下的局限性及优化方案，本节通过消融实验与代码重构，对 APF 的局部极小值陷阱和 Q-Learning 的稀疏奖励问题进行了深度剖析。

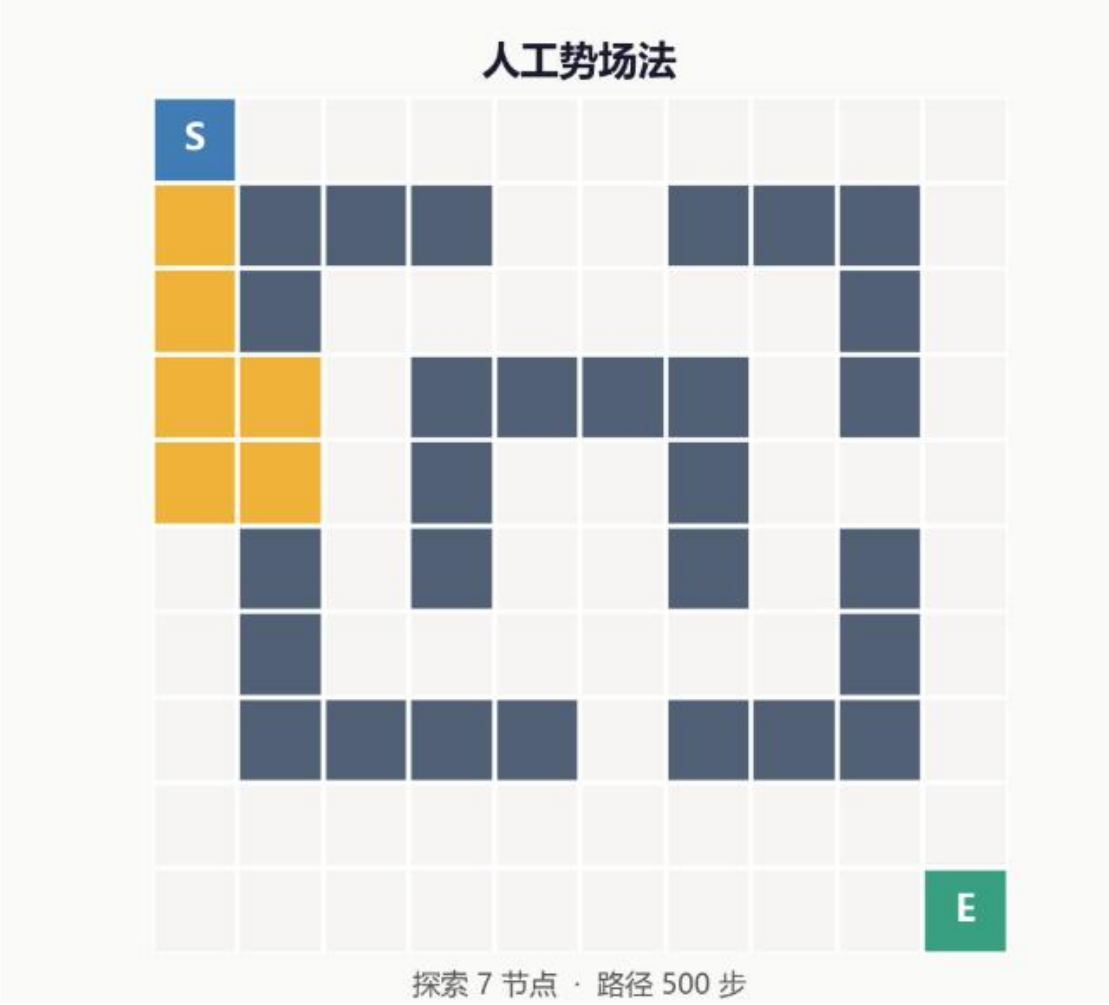
1. 人工势场法改进：随机扰动机制的消融分析

在应对高难度“凹字形”陷阱时，现有 APF 代码中人为植入了随机扰动逃逸机制（检测到轨迹震荡时强行注入随机位移）。为验证该机制的必要性，本实验移除了该段代码，恢复纯粹的物理受力模型，其结果极具警示意义。

核心代码：

```
if next_pos in visited_set and next_pos != self.end:
    # 检测到振荡（陷入局部极小值）
    # 简单处理：随机选一个合法邻居跳出
    directions = [(-1, 0), (1, 0), (0, -1), (0, 1)]
    candidates = [(current[0]+dr, current[1]+dc)
                  for dr, dc in directions
                  if 0 <= current[0]+dr < self.rows
                  and 0 <= current[1]+dc < self.cols
                  and self.grid[current[0]+dr][current[1]+dc] == 0]
    if candidates:
        next_pos = random.choice(candidates)
    else:
        print("  [APF] 陷入局部极小值且无法跳出。")
        return path, False
```

消融实验现象：去除随机扰动后，控制台显示 APF“达到最大迭代次数 500，未能到达终点”。可视化轨迹表明，在陷阱深处，向前的引力与三面墙壁的斥力绝对抵消，导致机器人在区区 7 个节点内陷入高达 500 步的高频无效死循环 。



如图，未探索到收敛路径。

2. 强化学习改进：势能奖励的消融实验

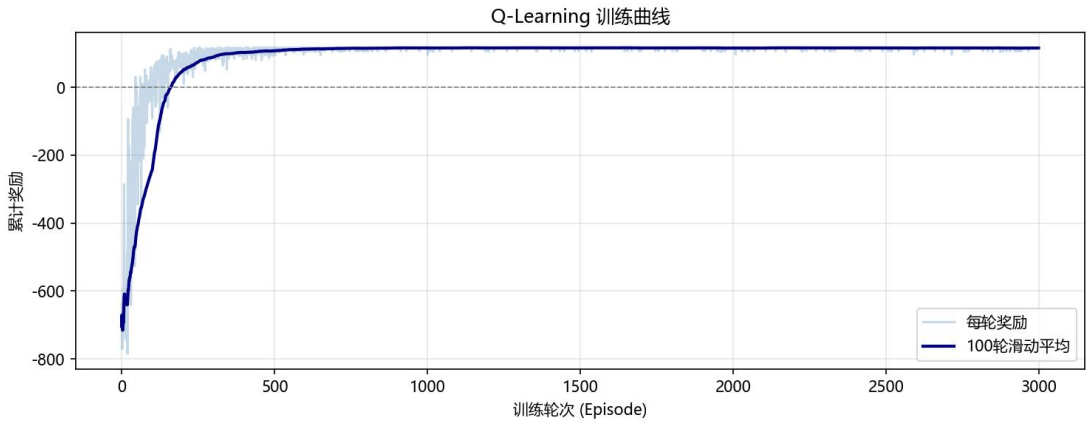
针对 Q-Learning 早期探索极度盲目、频繁撞墙的痛点，本实验重构了奖励函数。将原先的基础稀疏奖励（仅到达终点给 +100 分，其余为 -1 分）升级为了基于势能的稠密奖励（Potential-based Reward Shaping）：将曼哈顿距离的差值作为连续反馈注入系统，智能体每靠近终点一步额外加分，远离则倒扣分。

核心代码：

```python
# 核心改进：势能奖励塑形计算
if done:
    reward = 100
else:
    # 计算旧状态和新状态到终点的曼哈顿距离
    old_dist = abs(state[0] - self.end[0]) + abs(state[1] - self.end[1])
    new_dist = abs(next_state[0] - self.end[0]) + abs(next_state[1] - self.end[1])

    # 基础步数惩罚 -1，加上距离势能反馈（向终点走+2，远离终点-2）
    reward = -1 + 2.0 * (old_dist - new_dist)
```

对比实验现象：引入势能奖励后，训练日志表明早期（第 500 轮）的平均奖励直接从原来的 -36.4 跃升至 20.9；到第 1000 轮时，平均奖励高达 115.2（原版稀疏奖励此时仅为 80.1），并在后续轮次迅速稳定在 116.4 左右的高位。



我的反思：基础的 Q-Learning 在初始阶段完全是“盲人摸象”，在状态空间稍大时就会遭遇严重的冷启动灾难。将传统图搜索中的启发式思想（距离预估）通过 Reward Shaping 降维融入强化学习，本质上是给马尔可夫随机游走加上了“先验方向引力”。这不但将收敛速度提升了一倍以上，也让我深刻意识到：先验物理法则与无模型试错的结合，才是突破单一算法瓶颈的有效手段。未来在应对超大规模地图的指数爆炸问题时，必然需要引入深度强化学习（DQN）来替代庞大低效的 Q 表，而势能奖励塑形无疑将是加速此类神经网络收敛、实现工业级落地的底层基石。

七、 总结与展望

在本次路径规划算法的综合课程设计中，本实验跨越了图网络搜索、连续物理场模拟与马尔可夫决策过程，对 A*、BFS、Dijkstra、人工势场法（APF）以及强化学习进行了多维度的量化对比与消融剖析。通过在静态基础网格、复杂“凹字形”陷阱以及带有随机打滑机制的动态环境中进行实证检验，本报告得出以下核心科研结论与工程洞见：

1. 核心结论与算法生命周期边界

启发式偏置是压缩状态空间的绝对利器：在全局已知、确定且静态的环境下，A*算法通过曼哈顿距离注入目标导向偏置，以最低的算力消耗实现了对离散搜索树的强力剪枝，稳居此类场景的性能榜首。

脱离场景谈算法复杂度是工程大忌：在等权图网络中，Dijkstra 因维护优先队列而付出了无谓的 $O(\log N)$ 排序开销，暴露出高阶算法在均质环境中的性能降级现象；APF 虽具备 O(1) 的实时决策能力，但由于缺乏全局拓扑记忆，极易在死胡同中陷入驻点震荡。

“搜索”向“学习”的范式转移：传统搜索算法在面对底层物理运动具有不确定性（如底盘打滑、阵风干扰）的环境时，其追求的“名义最短路径”极其脆弱。强化学习通过数十万次的随机试错，将高昂的训练时间转化为极速的 O(1) 查表推理，成功学到了以空间换取生存率的马尔可夫防御策略，展示了最优控制理论在随机环境下的统治力。

2. 科学原理的反思 (No Free Lunch)

统观全部实验，没有任何一种算法能够打破“没有免费的午餐（No Free Lunch）”定理：

追求绝对的最优与极速（A*），必须以环境的绝对确定性和全局先验为代价。

追求毫秒级的动态反应（APF），必须承担丧失全局最优甚至死锁的风险。

追求对未知世界和随机干扰的完美掌控（RL），必须忍受前期漫长且极耗算力的试错折磨。

特别是在对 Q-Learning 引入势能奖励的消融实验中可以深刻看出：将传统图搜索的距离预估以降维形式注入无模型强化学习，是解决复杂系统冷启动灾难的最

优解。纯粹的物理定律（APF）、纯粹的几何运算（A）与纯粹的数据试错（RL）走向融合，是算法演进的必然规律。

7.3 未来展望与工业化延伸

受限于本次实验的离散栅格维度，当前系统仍属于表征层面的验证。在面向未来的实际机器人工程（如四足机器人野外巡检、高阶自动驾驶）中，本课题仍有以下极具价值的延伸方向：

多层级混合规划架构：将 A* 或 JPS（Jump Point Search）作为全局路径生成器提供宏观拓扑路点，底层接入引入了模拟退火机制的 APF 算法进行百赫兹级别的高频动态避障平滑。

向连续状态空间与深度强化学习演进：面对真实的毫米级连续物理世界，10×10 的离散 Q 表将遭遇维度爆炸。需引入 DQN （Deep Q-Network）或 DDPG 等深度强化学习算法，利用神经网络的非线性特征提取能力，实现端到端（End-to-End）的自主寻径与姿态控制。

动态环境下的增量式重规划：引入 D* Lite 或 LPA* 算法。当环境发生局部变化（如突然出现路障）时，仅在原有搜索树的基础上进行增量修复，而非全局重算，进一步逼近工业级规划器的严苛时延要求。

源代码：

```
# A* 算法
def heuristic(a, b, mode="manhattan"):
    """
    A* 启发函数 h(n)：估计当前节点到终点的代价。

    - Manhattan 距离：只允许上下左右移动时的最优选择
      h = |r1-r2| + |c1-c2|
    - Euclidean 距离：允许斜向移动时更准确

    这里采用 Manhattan 距离，因为地图只允许四方向移动。
    """
    if mode == "manhattan":
        return abs(a[0] - b[0]) + abs(a[1] - b[1])
    elif mode == "euclidean":
        return ((a[0] - b[0]) ** 2 + (a[1] - b[1]) ** 2) ** 0.5

def astar(grid, start, end):
    """
    A* 算法（A-Star Search）

    核心思路：
        f(n) = g(n) + h(n)
        g(n)：从起点到当前节点的实际代价（步数）
        h(n)：从当前节点到终点的启发估计代价（Manhattan 距离）

    通过 f 值优先级排序的最小堆（open_set）来决定下一个探索节点，
    使得算法优先探索"看起来更近终点"的方向，大幅减少无效搜索。
```

```
    参数:
        grid  : numpy 二维数组，0=可通行，1=障碍
        start : 起点坐标 (row, col)
        end   : 终点坐标 (row, col)

    返回:
        path          : 从起点到终点的最短路径列表，失败返回 None
        visited_order: 节点被访问的顺序列表（用于可视化）
    """
    rows, cols = grid.shape

    # open_set: 优先队列，元素为 (f_score, (row, col))
    open_set = []
    heapq.heappush(open_set, (0 + heuristic(start, end), start))
    '''
    什么是 heapq？ Python 里没有专门的堆数据类型，堆其实就是一个普通的列表 []。但是，我们绝对不能用原生的 append() 和 pop() 去操作它。
    必须把这个列表传给 heapq.heappush() 函数，这个函数会在底层用 C 语言自动帮你把元素塞到列表的特定位置，保证列表的第 0 个元素永远是最小的。
    为什么里面要塞一个元组 (f_score, start)？ 这是 Python 堆排序的一个极其关键的语法糖！
    当 heapq 在对元素进行排序找最小值时，如果存的是元组，它会默认按照元组的第 0 个元素（也就是 f_score）进行比大小。
    '''

    # came_from: 记录每个节点的前驱，用于路径回溯
    came_from = {}

    # g_score[node]: 从起点到 node 的实际步数代价
    g_score = {start: 0}

    # closed_set: 已完成评估的节点，不再重复处理
    closed_set = set()

    # 记录节点被弹出（正式访问）的顺序，用于后续动画可视化
    visited_order = []

    # 四方向：上、下、左、右
    directions = [(-1, 0), (1, 0), (0, -1), (0, 1)]

    while open_set:
        # 弹出 f 值最小的节点（最有希望的候选）
        _, current = heapq.heappop(open_set)
        '''
        heapq.heappop(open_set)：这行代码会以 O(\log N) 的极快速度，从堆里把 f 值（综合得分）最小的那个元组弹出来。
        这就是 A* 比 BFS 聪明的核心——它永远先挑离终点看起来最近的格子走。
        '''
        if current in closed_set:
            continue  # 已处理过，跳过（堆中可能有同一节点的旧记录）

        closed_set.add(current)
        visited_order.append(current)

        # 到达终点，回溯重建路径
        if current == end:
            path = []
            node = end
            while node in came_from:
                path.append(node)
                node = came_from[node]
            path.append(start)
            path.reverse()
            return path, visited_order

        # 遍历四个邻居
        for dr, dc in directions:
            nr, nc = current[0] + dr, current[1] + dc
            neighbor = (nr, nc)

            # 边界检查 + 障碍检查 + 是否已处理
            if not (0 <= nr < rows and 0 <= nc < cols):
                continue
            if grid[nr][nc] == 1:
                continue
            if neighbor in closed_set:
                continue

            # 计算经过 current 到达 neighbor 的实际代价（每步代价为 1）
            tentative_g = g_score[current] + 1

            # 若找到更优路径，更新记录
            if neighbor not in g_score or tentative_g < g_score[neighbor]:
                came_from[neighbor] = current
                g_score[neighbor] = tentative_g
                f = tentative_g + heuristic(neighbor, end)
                heapq.heappush(open_set, (f, neighbor))
```

```
    # open_set 耗尽仍未到达终点，说明无路可走
    return None, visited_order

# BFS 算法
def bfs(grid, start, end):
    """
    BFS 算法（广度优先搜索，Breadth-First Search）

    核心思路：
        从起点出发，按"距起点步数"由近到远逐层展开搜索。
        先访问所有距起点 1 步的节点，再访问距起点 2 步的，以此类推。
        由于每步代价相同（均为 1），BFS 第一次到达终点时
        走过的路径必然是步数最少的路径，因此 BFS 保证最优性。

    与 A* 的核心区别：
        BFS 没有启发函数，不区分方向好坏，向四周"无脑"展开。
        A* 利用启发函数"有目的"地搜索，通常访问节点更少。

    参数 & 返回值同 astar()
    """
    rows, cols = grid.shape

    # 使用 deque 实现队列（FIFO），保证按层扩展，deque 是基于双向链表实现的，它的 popleft() 操作是严格的 O(1) 时间复杂度。这是保证 BFS 算法在大地图上不卡死的物理基础。
    queue = deque([start])

    # visited: 已加入队列的节点，防止重复入队
    visited = {start}

    # came_from: 记录路径，start 的前驱记为 None
    came_from = {start: None}

    visited_order = []
    directions = [(-1, 0), (1, 0), (0, -1), (0, 1)]

    while queue:
        current = queue.popleft()
        visited_order.append(current)

        # 到达终点，回溯路径
        if current == end:
            path = []
            node = end
            while node is not None:
                path.append(node)
                node = came_from[node]
            path.reverse()
            return path, visited_order

        for dr, dc in directions:
            nr, nc = current[0] + dr, current[1] + dc
            neighbor = (nr, nc)

            if not (0 <= nr < rows and 0 <= nc < cols): # 越界剪枝
                continue
            if grid[nr][nc] == 1: # 碰撞剪枝
                continue
            if neighbor in visited: # 防环剪枝（及其重要！！！防止陷入死循环）
                continue

            visited.add(neighbor)
            came_from[neighbor] = current
            queue.append(neighbor)

    return None, visited_order


# Dijkstra 算法
def dijkstra(grid, start, end):
    """
    Dijkstra 算法

    核心思路：
        本质上是 BFS 的"加权版本"。维护一个"到起点最短距离"的字典，
        每次从未访问节点中选择距离最小的节点进行扩展。

        当地图所有边权重相同（如本实验每步代价均为 1）时，
        Dijkstra 和 BFS 的结果完全一致，效率也相近。

        Dijkstra 的优势体现在边权不同的情况（如地形代价不同），
        A* 可以看作是 Dijkstra + 启发函数的结合。

    局限性：
        没有启发函数，会向四周均匀扩展，在大型地图上效率低于 A*。
        对于负权边（如本题不存在）会失效。

    参数 & 返回值同 astar()
    """
```

```
        rows, cols = grid.shape

        # dist: 起点到各节点的最短距离，初始为无穷大
        dist = {start: 0}
        came_from = {}
        # 优先队列：(距离, 节点)
        pq = [(0, start)]
        closed_set = set()
        visited_order = []

        directions = [(-1, 0), (1, 0), (0, -1), (0, 1)]

        while pq:
            d, current = heapq.heappop(pq) # 从堆里弹出当前所有已知边界中，累积步数 d 最小的那个节点。
            # 元组解包。我们拿到了当前节点坐标 current，以及走到它的代价 d。

            if current in closed_set:
                continue

            closed_set.add(current)
            visited_order.append(current)

            if current == end:
                path = []
                node = end
                while node in came_from:
                    path.append(node)
                    node = came_from[node]
                path.append(start)
                path.reverse()
                return path, visited_order

            for dr, dc in directions:
                nr, nc = current[0] + dr, current[1] + dc
                neighbor = (nr, nc)

                if not (0 <= nr < rows and 0 <= nc < cols):
                    continue
                if grid[nr][nc] == 1:
                    continue
                if neighbor in closed_set:
                    continue

                new_dist = dist[current] + 1  # 本实验边权均为1
                if neighbor not in dist or new_dist < dist[neighbor]:
                    dist[neighbor] = new_dist
                    came_from[neighbor] = current
                    heapq.heappush(pq, (new_dist, neighbor))

        return None, visited_order


# 人工势场法
# 思路来源：把终点看成"引力场"，障碍看成"斥力场"，
# 机器人在合力方向上逐步移动，直到到达终点。

class ArtificialPotentialField:
    """
    人工势场法（APF）

    核心公式：
        引力 F_att = k_att * (goal - current)
        斥力 F_rep = k_rep * (1/d - 1/d0) * (1/d^2) * 方向向量
                    当 d < d0 时生效，d 为到障碍的距离，d0 为斥力影响范围
        合力 = F_att + sum(F_rep)

    优点：实时性好，适合动态环境（障碍可以移动）
    缺点：
        1. 容易陷入局部极小值（某处引力斥力恰好平衡）
        2. 路径不一定最短
        3. 在狭窄通道中可能震荡
    """

    def __init__(self, grid, start, end,
                 k_att=1.0,     # 引力系数
                 k_rep=2.0,     # 斥力系数
                 d0=2.5,        # 斥力有效影响范围（格子数）
                 step_size=1,  # 每步移动1格
                 max_iter=500):# 最大迭代步数（防止死循环）

        self.grid = grid
        self.start = start
        self.end = end
        self.k_att = k_att
        self.k_rep = k_rep
        self.d0 = d0
        self.step_size = step_size
```

```
    self.max_iter = max_iter

    self.rows, self.cols = grid.shape

    # 预计算所有障碍格子的坐标，避免每步重复遍历地图
    self.obstacles = [(r, c) for r in range(self.rows)
                      for c in range(self.cols) if grid[r][c] == 1]

def _attractive_force(self, current):
    """
    计算引力：指向终点的向量。
    F_att = k_att * (goal_pos - current_pos)
    归一化处理，保证每步移动量一致。
    """
    dr = self.end[0] - current[0]
    dc = self.end[1] - current[1]
    dist = (dr ** 2 + dc ** 2) ** 0.5
    if dist < 1e-6:
        return np.array([0.0, 0.0])
    return self.k_att * np.array([dr, dc]) / dist

def _repulsive_force(self, current):
    """
    计算斥力：所有在影响范围内障碍的斥力之和。
    斥力方向：从障碍指向当前位置（推开机器人）
    斥力大小：距离越近，斥力越大
    """
    F_rep = np.array([0.0, 0.0])

    for obs in self.obstacles:
        dr = current[0] - obs[0]
        dc = current[1] - obs[1]
        d = (dr ** 2 + dc ** 2) ** 0.5

        if d < 1e-6 or d > self.d0:
            continue  # 太近（在障碍内）或超出影响范围，跳过

        # 斥力公式：k_rep * (1/d - 1/d0) / d^2 * 方向向量
        magnitude = self.k_rep * (1.0 / d - 1.0 / self.d0) / (d ** 2)
        F_rep += magnitude * np.array([dr, dc]) / d

    return F_rep

def _get_best_neighbor(self, current, force):
    """
    将连续势场力映射到离散格子：
    找四个邻居中，与合力方向最接近（点积最大）的合法格子。
    """
    directions = [(-1, 0), (1, 0), (0, -1), (0, 1)]
    best_neighbor = None
    best_score = -float('inf')

    for dr, dc in directions:
        nr, nc = current[0] + dr, current[1] + dc
        if not (0 <= nr < self.rows and 0 <= nc < self.cols):
            continue
        if self.grid[nr][nc] == 1:
            continue

        # 计算邻居方向向量与合力的点积（越大越符合力的方向）
        move_dir = np.array([dr, dc], dtype=float)
        score = np.dot(force, move_dir)
        if score > best_score:
            best_score = score
            best_neighbor = (nr, nc)

    return best_neighbor

def run(self):
    """
    执行人工势场法寻径。

    返回:
        path   : 路径节点列表
        success: 是否成功到达终点
    """
    current = self.start
    path = [current]
    visited_set = {current}  # 用于检测振荡（已访问节点重复出现）

    for _ in range(self.max_iter):
        if current == self.end:
            return path, True

        # 计算合力
        F_att = self._attractive_force(current)
        F_rep = self._repulsive_force(current)
        F_total = F_att + F_rep

        # 选择最优邻居
```

```
            next_pos = self._get_best_neighbor(current, F_total)

            if next_pos is None:
                # 所有邻居都是障碍，被困住了
                print("  [APF] 机器人被障碍包围，无法移动。")
                return path, False

            if next_pos in visited_set and next_pos != self.end:
                # 检测到振荡（陷入局部极小值）
                # 简单处理：随机选一个合法邻居跳出
                directions = [(-1, 0), (1, 0), (0, -1), (0, 1)]
                candidates = [(current[0]+dr, current[1]+dc)
                              for dr, dc in directions
                              if 0 <= current[0]+dr < self.rows
                              and 0 <= current[1]+dc < self.cols
                              and self.grid[current[0]+dr][current[1]+dc]
== 0]
                if candidates:
                    next_pos = random.choice(candidates)
                else:
                    print("  [APF] 陷入局部极小值且无法跳出。")
                    return path, False

            current = next_pos
            path.append(current)
            visited_set.add(current)

        print(f"  [APF] 达到最大迭代次数 {self.max_iter}，未能到达终点。
")
        return path, False


# Q-Learning 强化学习
class QLearningAgent:
    """
    Q-Learning 强化学习寻径

    原理：
        将地图中每个可通行格子视为"状态"，上下左右四个方向为"动作"。
        Agent 在地图上反复探索，每次到达终点获得正奖励，
        撞障碍或走重复路获得负奖励。
        经过大量训练（episode），Q 表收敛，
        最终 Agent 能根据 Q 值贪心地找到较优路径。

    这是强化学习（RL）的最基础形式，属于无模型（model-free）方法。
    Agent 一开始完全不知道地图结构，完全靠"试错"积累经验。

    核心公式（Q 值更新，Bellman 方程）：
        Q(s,a) ← Q(s,a) + α * [r + γ * max_a' Q(s',a') - Q(s,a)]

        其中：
        α (alpha)  = 学习率（0~1），控制新信息覆盖旧信息的程度
        γ (gamma)  = 折扣因子（0~1），控制对未来奖励的重视程度
        r          = 即时奖励
        s'         = 执行动作后到达的新状态
    """

    def __init__(self, grid, start, end,
                 alpha=0.1,      # 学习率
                 gamma=0.9,      # 折扣因子
                 epsilon=1.0,    # 初始探索率（ε-贪婪策略）
                 epsilon_min=0.01,
                 epsilon_decay=0.995,
                 n_episodes=5000,
                 max_steps=200):

        self.grid = grid
        self.start = start
        self.end = end
        self.alpha = alpha
        self.gamma = gamma
        self.epsilon = epsilon
        self.epsilon_min = epsilon_min
        self.epsilon_decay = epsilon_decay
        self.n_episodes = n_episodes
        self.max_steps = max_steps

        self.rows, self.cols = grid.shape
        self.actions = [(-1,0),(1,0),(0,-1),(0,1)]  # 上下左右

        # Q 表：状态 (r,c) × 动作 4 个方向
        # 初始化为 0
        self.Q = {}
        for r in range(self.rows):
            for c in range(self.cols):
                if grid[r][c] == 0:
                    self.Q[(r, c)] = [0.0] * 4

        self.episode_rewards = []  # 记录训练过程
```

```
def _reward(self, state, next_state, done):
    """
    奖励函数设计：
    - 到达终点：+100（强正奖励，激励 Agent 找到终点）
    - 撞墙或越界：-10（惩罚无效动作）
    - 普通移动：-1（轻微惩罚，激励 Agent 走最短路）
    """
    if next_state == self.end:
        return 100
    if next_state is None:
        return -10
    return -1

def _step(self, state, action_idx):
    """
    执行一个动作，返回 (新状态, 奖励, 是否结束)。
    若动作撞墙/越界，原地不动，给负奖励。
    """
    dr, dc = self.actions[action_idx]
    nr, nc = state[0] + dr, state[1] + dc

    # 边界或障碍检查
    if not (0 <= nr < self.rows and 0 <= nc < self.cols):
        return state, -10, False  # 越界，原地
    if self.grid[nr][nc] == 1:
        return state, -10, False  # 撞墙，原地

    next_state = (nr, nc)
    done = (next_state == self.end)
    reward = 100 if done else -1
    return next_state, reward, done

def _choose_action(self, state):
    """
    ε-贪婪策略：
    - 以 ε 的概率随机探索（避免陷入局部最优）
    - 以 1-ε 的概率选择当前 Q 值最大的动作（利用已有知识）
    训练初期 ε 大（多探索），后期 ε 小（多利用）。
    """
    if random.random() < self.epsilon:
        return random.randint(0, 3)
    return int(np.argmax(self.Q[state]))

def train(self):
    """训练 Q-Learning Agent"""
    print(f" 开始训练 Q-Learning，共 {self.n_episodes} 个 episode...")

    for ep in range(self.n_episodes):
        state = self.start
        total_reward = 0

        for _ in range(self.max_steps):
            action = self._choose_action(state)
            next_state, reward, done = self._step(state, action)
            total_reward += reward

            # Q 值更新（Bellman 方程）
            old_q = self.Q[state][action]
            if next_state in self.Q:
                max_next_q = max(self.Q[next_state])
            else:
                max_next_q = 0

            self.Q[state][action] = old_q + self.alpha * (
                reward + self.gamma * max_next_q - old_q
            )

            state = next_state

            if done:
                break

        self.episode_rewards.append(total_reward)

        # ε 衰减
        if self.epsilon > self.epsilon_min:
            self.epsilon *= self.epsilon_decay

        if (ep + 1) % 500 == 0:
            avg_reward = np.mean(self.episode_rewards[-500:])
            print(f"  Episode {ep+1}/{self.n_episodes}  "
                  f"ε={self.epsilon:.3f}  近500轮平均奖励={avg_reward:.1f}")

    print(" 训练完成！")

def get_path(self):
    """
    训练后，用贪婪策略（始终选 Q 值最大动作）推导最终路径。
```

```
"""
state = self.start
path = [state]
visited = {state}

for _ in range(self.max_steps):
    if state == self.end:
        return path
    if state not in self.Q:
        break

    action = int(np.argmax(self.Q[state]))
    dr, dc = self.actions[action]
    nr, nc = state[0] + dr, state[1] + dc
    next_state = (nr, nc)

    if next_state in visited or self.grid[nr][nc] == 1:
        break  # 陷入循环或撞墙，训练可能还不充分

    path.append(next_state)
    visited.add(next_state)
    state = next_state

return path
```

```
def plot_training_curve(self, save_path=None):
    """绘制训练奖励曲线，观察 Agent 的学习进度"""
    fig, ax = plt.subplots(figsize=(10, 4))

    # 平滑处理：每 100 个 episode 取平均
    window = 100
    smoothed = [np.mean(self.episode_rewards[max(0, i-window):i+1])
                for i in range(len(self.episode_rewards))]

    ax.plot(self.episode_rewards, alpha=0.3, color='steelblue',
label='每轮奖励')
    ax.plot(smoothed, color='navy', linewidth=2, label=f'{window}轮
滑动平均')
    ax.axhline(y=0, color='gray', linestyle='--', linewidth=0.8)
    ax.set_xlabel('训练轮次 (Episode)')
    ax.set_ylabel('累计奖励')
    ax.set_title('Q-Learning 训练曲线')
    ax.legend()
    ax.grid(True, alpha=0.3)

    plt.tight_layout()
    if save_path:
        plt.savefig(save_path, dpi=150)
    plt.show()
```